UNIVERSITY OF PISA

Lesson 8

Silvia Giulia Galfrè

Part 2

PHYLOGENETIC ANALYSIS

2. Building phylogenetic trees with maximum parsimony

Align sequences (MSA).

Define informative columns

Build the tree that require the minimum number of mutations

2.1 – Simplest solution

- Enumerate all possible trees and their costs
- NP-hard
- Possible only for less than 10 seq.

2.2 – *Branch and bound*

- The idea: try building trees step-by-step; any partial tree that already can't beat the best score you've seen is thrown away early.
- The three words
  - Branch: grow alternatives (add the next taxon in every possible place to make many partial trees).
  - Bound: for each partial tree, compute a best-case minimum number of steps it could ever achieve when finished.
  - Prune: if that best-case is not better than your current best full-tree score, stop exploring that branch.

2.2 – *Branch and bound*

- How it runs
  1. Get a starting best score (e.g., from a quick heuristic tree).
  2. Start with a tiny tree (3–4 taxa).
  3. Add one taxon in all possible places → you get several partial trees (branch).
  4. For each partial tree, quickly estimate the lowest possible steps it could end with (bound). What is the **minimum** number of steps this partial tree is already forced to have, no matter how we place the remaining taxa?
  5. If that number ≥ current best score, discard it (prune).
  6. Otherwise, keep growing it.
  7. When a full tree is built, if it's shorter, update the best score.
  8. Continue until all branches are either finished or pruned. The trees that remain with the best score are your most parsimonious trees.

3. Building phylogenetic trees with maximum likelihood

- It estimate the likelihood of possible trees by multiply the estimated probability of each event implied in the tree.
- It can use different models of DNA evolution
- It is quite computationally intensive (NP-hard)
- To reduce the complexity: pruning algorithm (computing the likelihood of sub-trees)

Building a phylogenetic tree

- Web tools available from NCBI, EBI, genomes.jp
  - https://www.ncbi.nlm.nih.gov/tools/cobalt/re_cobalt.cgi
  - https://www.ebi.ac.uk/Tools/phylogeny/simple_phylogeny/
  - https://www.genome.jp/tools-bin/ete
- Desktop or command line tools also available
  - PhyML: http://www.atgc-montpellier.fr/phyml/download.php
- Biopython has wrappers to invoke command line tools

Visualising a phylogenetic tree

- Web viewers from from NCBI, EBI, genomes.jp
- External tools:
  - Export tree (various formats, a common one is Newick .nwk) then import in external tool
    - TreeViewer https://treeviewer.org/
  - Interactive tree of life: https://itol.embl.de/
  - Biopython Bio.Phylo module

Bio.Phylo

- Functions to read and write trees, similar to AlignIO, SeqIO

```
import Bio
import Bio.Phylo as Phylo
```

Bio.Phylo

- Functions to display trees

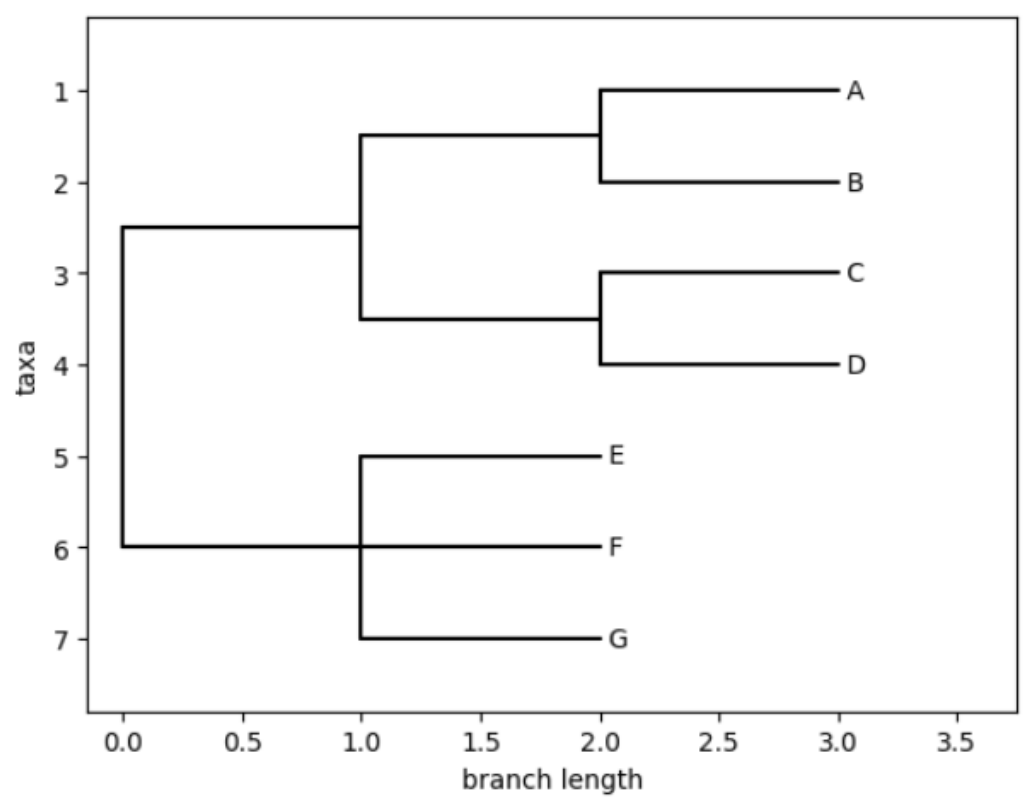
```
tree = Phylo.read('/content/drive/MyDrive/Colab Notebooks/simple.dnd', 'newick')
print(tree)
Phylo.draw_ascii(tree)
```

```
Tree(rooted=False, weight=1.0)
    Clade()
        Clade()
            Clade()
                Clade(name='A')
                Clade(name='B')
            Clade()
                Clade(name='C')
                Clade(name='D')
        Clade()
            Clade(name='E')
            Clade(name='F')
            Clade(name='G')
                                                  ________________________ A
                             ________________________|
                            |                        |________________________ B
  ______________________________|
 |                          |                        ________________________ C
 |                          |________________________|
_|                                                   |________________________ D
 |
 |                           ________________________ E
 |                          |
 |__________________________|________________________ F
                            |
                            |________________________ G
```

Bio.Phylo

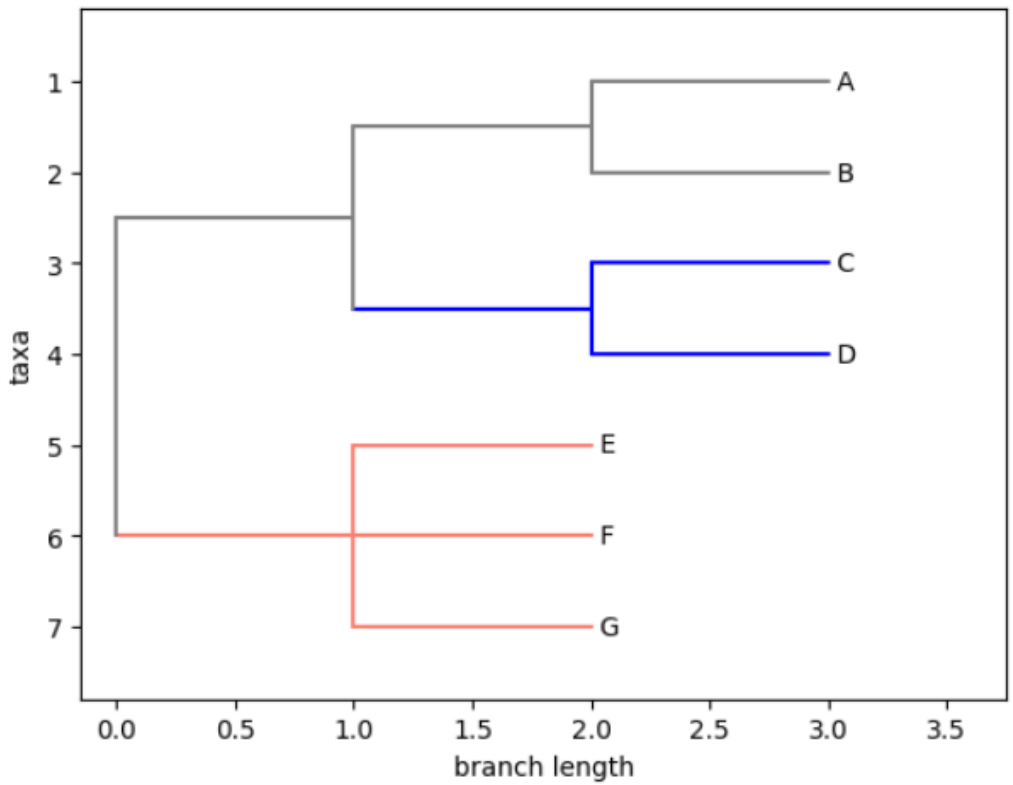
- Functions to display trees

```
Phylo.draw(tree)
```



Bio.Phylo

```
treep.root.color = 'gray'
mrca = treep.common_ancestor({"name": "E"},{"name": "F"})
mrca.color = 'salmon'
treep.clade[0,1].color = 'blue'
Phylo.draw(treep)
```



Bio.Phylo

- Functions to invoke command line tools
- Module `Bio.Phylo.Applications`
- https://biopython.org/docs/1.76/api/Bio.Phylo.Applications.html
- E.g. `PhymlCommandline`
  - wrapper for PhyML
  - requires PhyML to be installed (downloaded)

Homework

Build a phylogenetic tree for the Hemoglobin protein sequences (file *hemoglobin_proteins.fasta*)

Try different tools and different visualisation methods.

MOTIF DISCOVERY ALGORITHMS

Problem definition

- *Motif* – a non-trivial sequential pattern shared across multiple sequences
- It should have a minimum relevant length
- Why it is important?
  - I observe a common feature (e.g. common regulatory control)
  - If I find a common motif that region could be the source for the common feature

Motifs classes

- Mainly two:
  - *Deterministic*: the name indicates that it can be present or absent in the sequence. These are lake the protein motifs from prosite database.
  - *Probabilistic:* the motif model is much less defined… it contains more variability. For example, a protein can have a certain probability to contains a specific motif.

Homework

Search for all protein sequences of orthologues for BCL6.

Build a phylogenetic tree. Try to observe evolutionary relationships and possible protein motifs.